

PROGETTO RETI LOGICHE - Matteo Mancin

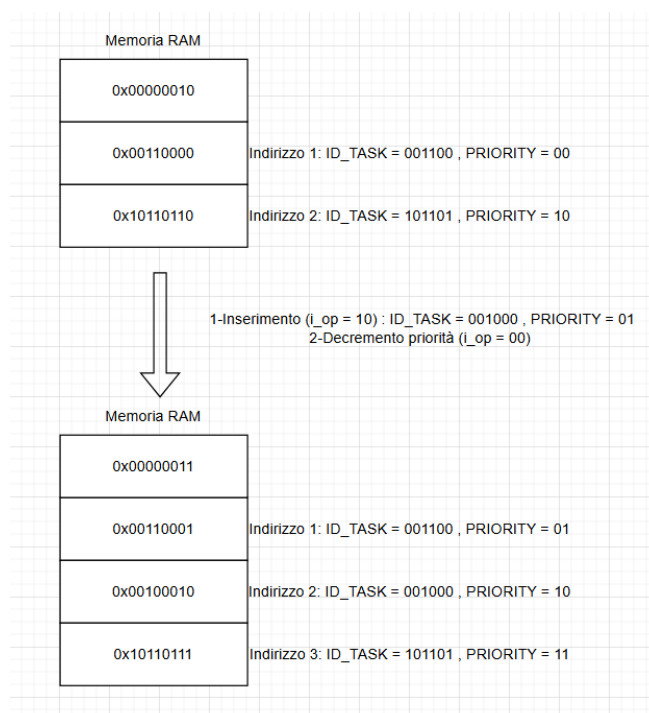
1. Obiettivo

L'obiettivo del progetto di reti logiche è la progettazione, descrizione in linguaggio VHDL e sintesi di un modulo hardware sincrono che opera come un gestore di una coda a priorità (Priority Queue - Task Manager/Scheduler). Il sistema si interfaccia con una memoria RAM esterna, utilizzandola per mantenere e manipolare una lista continua e ordinata di task.

Ogni Task è rappresentato da un singolo byte contenente due campi informativi: un identificativo univoco a 6 bit (ID_TASK) e un livello di importanza a 2 bit (PRIORITY, dove il valore 0 rappresenta la priorità massima e 3 la minima). La particolarità e la sfida architetturale del sistema risiedono nella gestione dinamica della memoria: la rete logica deve preservare la continuità e l'ordinamento dell'array hardware, gestendo autonomamente gli "shift" dei dati in memoria senza l'ausilio di un processore software. All'indirizzo 0 della memoria viene costantemente mantenuto e aggiornato il numero totale di task validi presenti, mentre gli indirizzi successivi ospitano la coda effettiva.

Il modulo comunica con l'esterno tramite un protocollo di *handshake* basato sui segnali *i_start* e *o_done* ed è in grado di eseguire quattro micro-istruzioni (*i_op*):

- Svuotamento totale della coda.
- Inserimento di un nuovo Task (con verifica preventiva di non-duplicazione e inserimento stabile in coda ai pari-priorità).
- Rimozione del Task a priorità maggiore (in testa alla coda) con relativa compattazione della memoria.
- Aggiornamento (decremento di importanza) della priorità di tutti i Task presenti, con gestione della saturazione al valore minimo.



Nell'esempio si vede come viene modificata la memoria RAM dopo un inserimento e un decremento di priorità: inizialmente la lista conteneva 2 task di priorità 0 e 2 : dopo le 2 operazioni la lista contiene 3 task (indirizzo 0 aggiornato a 0x0..011) , il nuovo task ha aveva priorità 1 quindi è stato inserito tra i due che erano già presenti. Infine l'operazione di decremento di priorità ha portato i task ad avere rispettivamente :1, 2 e 3 .

2. Architettura

Il modulo *project_reti_logiche* è stato implementato utilizzando una singola Macchina a Stati Finiti (FSM) che gestisce sia il flusso di controllo delle operazioni sia l'interfacciamento con la memoria esterna. Dal punto di vista implementativo, l'architettura si basa su un approccio standard a due processi: un processo sincrono, sensibile al fronte di salita del segnale *i_clk* e al segnale asincrono *i_rst*, deputato all'aggiornamento dello stato corrente; e un processo combinatorio, che calcola lo stato successivo e definisce i valori dei segnali in uscita.

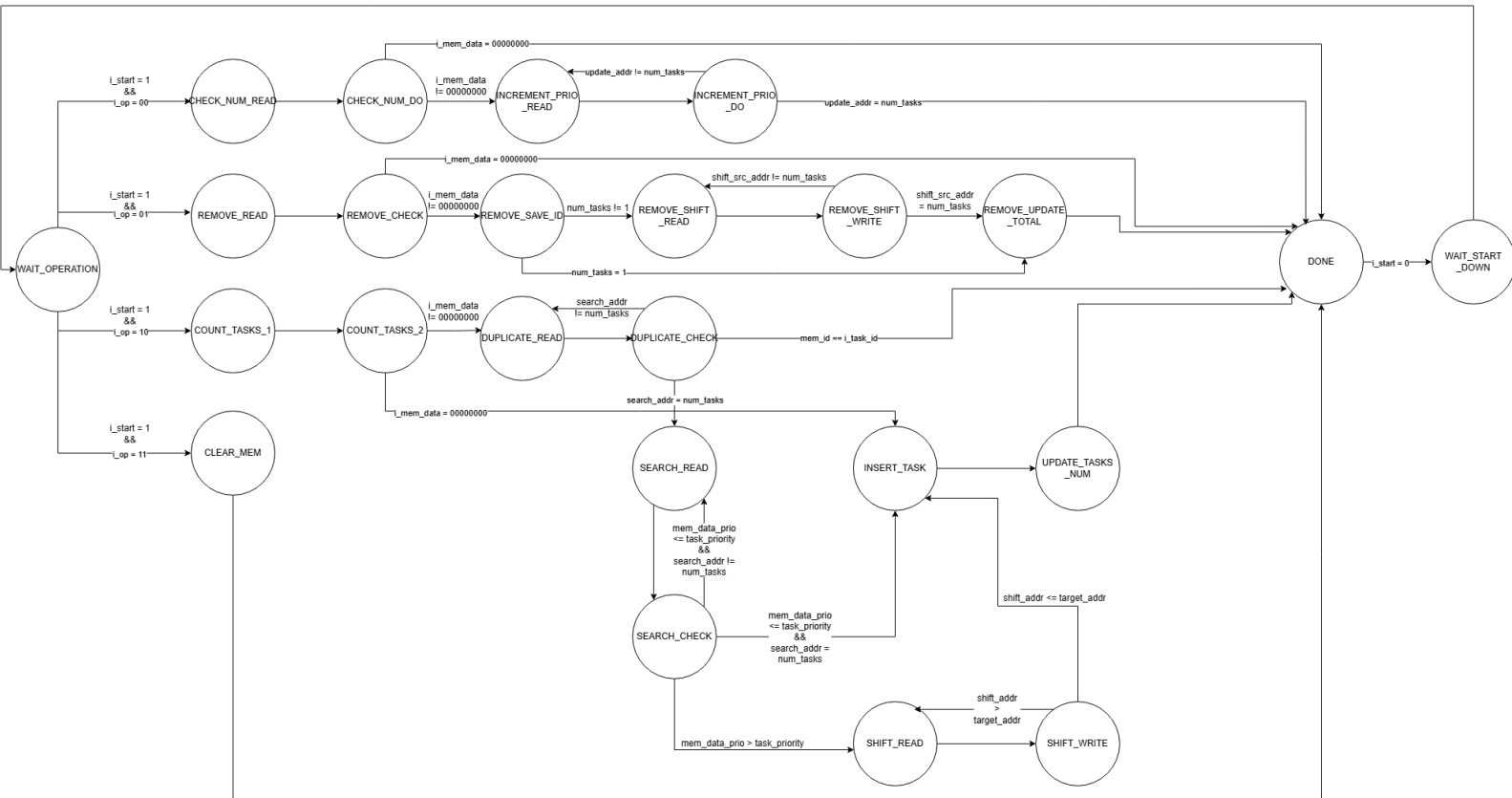
La Macchina a Stati è stata progettata per garantire il rigido rispetto del protocollo di handshake basato sui segnali *i_start* e *o_done*.

Sebbene la descrizione VHDL sia racchiusa in un'unica architettura comportamentale, il sistema modella a tutti gli effetti una FSMD (Finite State Machine with Datapath). Lo schema funzionale della rete può essere concettualmente diviso in due macro-moduli strettamente interconnessi, che riflettono la natura dei due processi software:

- **Modulo 1 : rete sequenziale** (memoria di stato e registri dei dati):
Questo modulo rappresenta la componente di memoria fisica del circuito (banchi e flip-flop). E' governato esclusivamente dal processo sincrono e ha il compito di campionare e stabilizzare i segnali ad ogni colpo di clock.
Al suo interno, il modulo implementa:
 - Il Registro di Stato: memorizza lo stato attuale della FSM (*current_state*), garantendo l'evoluzione dell'algoritmo
 - I Registri operativi (*datapath*): memorizzano temporaneamente i dati estratti dalla RAM o calcolati dalla logica. Per garantire una corretta sintesi senza la generazione di Latch indesiderati, l'architettura fa un uso rigoroso del pattern *curr_ / next_*. Il modulo sequenziale si limita a ricopiare i valori di ingresso (es: *next_num_tasks*, *next_shift_addr*) sulle rispettive uscite stabili (es: *curr_num_tasks*, *curr_shift_addr*).
 - Reset Asincrono: in caso di assenza del segnale *i_rst* = '1', questo modulo forza istantaneamente l'azzeramento di tutti i puntatori interni e riporta la FSM allo stato di *RESET_INIT*, indipendentemente dal clock.
- **Modulo 2 : rete combinatoria** (control unit e rete operativa):
Questo modulo, implementato tramite il processo combinatorio, contiene tutta l'intelligenza logico-aritmetica del sistema. Legge gli ingressi esterni (*i_op*, *i_task_id*, *i_mem_data*) e i segnali interni stabilizzati dal Modulo 1, per calcolare istantaneamente le uscite. Assolve due funzioni fondamentali:
 - Logica di Controllo (Control Unit): tramite un costrutto *case*, decodifica l'operazione richiesta in fase di *WAIT_OPERATION* e gestisce l'evoluzione dei 26 stati della FSM. Genera inoltre i segnali di abilitazione per la comunicazione con la RAM (*o_mem_en*, *o_mem_we*),
 - Logica Operativa (*Datapath* inferito): genera le reti logiche per l'elaborazione dei dati: ad esempio le istruzioni di incremento e decremento dei puntatori istanziano fisicamente dei sommatore / sottrattori hardware; le assegnazioni condizionali (*if / else*) per la verifica delle priorità o la ricerca dei duplicati

inferiscono comparatori e multiplexer che instradano il dato corretto verso il bus o_mem_data

Il funzionamento del sistema si basa sul continuo scambio di dati tra questi due moduli. La rete combinatoria valuta lo stato attuale current_state e i registri correnti (curr_...) forniti dalla rete sequenziale per calcolare le operazioni matematiche e le scelte logiche. Il risultato di queste operazioni viene emesso sui segnali “futuri” (next_state, next_addr). Al successivo fronte di salita del clock, la rete sequenziale cattura questi nuovi valori e aggiorna le proprie uscite, permettendo al sistema di avanzare al ciclo di elaborazione successivo.



Gli stati della macchina a stati sono questi (divisi per operazione per leggibilità):

- **WAIT_OPERATION:** È lo stato di attesa principale. Il modulo rimane in questo stato disabilitando i segnali verso la memoria, finché la logica esterna non porta i_start a 1. A questo punto, il modulo campiona il segnale i_op per smistare l'esecuzione.
- **RESET_INIT / RESET_WAIT:** Questi stati gestiscono la condizione di reset asincrono. All'attivazione di i_rst, la FSM transita in RESET_INIT, dove provvede a svuotare la lista scrivendo il valore 0x00 all'indirizzo 0 della memoria, mantenendo o_done alto per indicare la non disponibilità del modulo. Al rilascio del reset, si passa a RESET_WAIT per abbassare il segnale o_done e prepararsi al normale funzionamento.

OPERAZIONE 00 - decremento priorità di tutti i task:

- **CHECK_NUM_READ / CHECK_NUM_DO** : Prima di iniziare la vera e propria operazione di riduzione di priorità di tutti i task, CHECK_NUM_READ/DO controllano che la lista non sia vuota; in caso lo sia transitano direttamente allo stato DONE in quanto non servono ulteriori operazioni, in caso non lo sia transitano allo stato INCREMENT_PRIO_READ
- **INCREMENT_PRIO_READ / INCREMENT_PRIO_DO** : Questi stati sono dedicati all'incremento dei bit dedicati alla priorità di tutti i task nella lista (di fatto diminuendo la priorità logica). Selezionano tramite curr_update_addr l'indirizzo da incrementare, si selezionano gli ultimi 2 bit: se sono uguali a 11 (=3 dec) non si fa niente in quanto sono già a priorità minima, mentre in tutti gli altri casi si aumenta di 1. Dopo questa operazione lo stato INCREMENT_PRIO_DO controlla se siamo arrivati in fondo alla lista, in tal caso si transita nello stato DONE, in caso contrario si incrementa il segnale next_update_addr e si ritorna nello stato INCREMENT_PRIO_READ.

OPERAZIONE 01 - Rimozione del task in testa alla lista

- **REMOVE_READ / REMOVE_CHECK**: Il sistema verifica che la lista contenga almeno un task leggendo il totale all'indirizzo 0. In caso di lista vuota, il segnale next_task_id_out viene forzato al valore binario "000000" (zero standard per l'ID) e la FSM transita direttamente nello stato DONE. Se è presente almeno un task, si imposta il puntatore di memoria al primo elemento (o_mem_addr = 1) e si transita in REMOVE_SAVE_ID.
- **REMOVE_SAVE_ID**: Questo stato campiona l'ID del task da rimuovere (i 6 bit più significativi letti) e lo salva nel registro di uscita next_task_id_out. Se la lista conteneva un solo elemento, non è necessario alcuno shift in memoria, pertanto si transita direttamente a REMOVE_UPDATE_TOTAL. Se sono presenti più task, il sistema prepara lo "shift in avanti" inizializzando due puntatori: next_shift_src_addr a 2 (il primo elemento da spostare) e next_shift_dst_addr a 1 (la posizione vuota da riempire). Si passa poi a REMOVE_SHIFT_READ.
- **REMOVE_SHIFT_READ / REMOVE_SHIFT_WRITE**: Questo loop esegue la compattazione della memoria. Nello stato di lettura, viene prelevato il dato puntato da curr_shift_src_addr; nello stato di scrittura, tale dato viene immediatamente sovrascritto all'indirizzo curr_shift_dst_addr. Successivamente, entrambi i puntatori vengono incrementati di 1. Se il puntatore sorgente raggiunge il numero originario dei task, lo shift della lista è concluso e si passa a REMOVE_UPDATE_TOTAL; in caso contrario, il ciclo riprende da REMOVE_SHIFT_READ.
- **REMOVE_UPDATE_TOTAL**: Conclusa la traslazione dei dati, la FSM si riposiziona sull'indirizzo 0 e vi sovrascrive il numero totale dei task decrementato di 1. Questo garantisce la coerenza tra il contatore e la memoria effettiva. Infine, si transita allo stato DONE.

OPERAZIONE 10 - Inserimento di un nuovo task

- **COUNT_TASKS_1 / COUNT_TASKS_2**: Il sistema interroga l'indirizzo 0 per ottenere il numero totale di task attualmente presenti. Se la lista è completamente vuota, la FSM deduce che non ci sono duplicati né spostamenti da fare: elegge l'indirizzo 1 come next_target_addr e salta direttamente a INSERT_TASK. Se la lista è popolata, inizializza un puntatore di ricerca all'indirizzo 1 e transita allo stato DUPLICATE_READ.

- **DUPLICATE_READ / DUPLICATE_CHECK:** Questo loop realizza la prima scansione totale della lista per garantire l'univocità dell'ID. Dopo aver letto il byte in memoria, i 6 bit più significativi vengono estratti e confrontati con l'i_task_id in ingresso. Se si rileva un conflitto (ID identico), l'operazione viene annullata transitando in DONE. Se l'ID è univoco e si è raggiunta la fine della lista, il puntatore viene riavvolto a 1 per la fase successiva; altrimenti, il loop prosegue incrementando l'indirizzo.
- **SEARCH_READ / SEARCH_CHECK:** Questo secondo loop individua il punto esatto di inserimento. Il sistema estrae i 2 bit di priorità del task letto e li confronta con la priorità di quello da inserire. Finché la priorità in memoria è numericamente minore o uguale, il loop scorre in avanti, garantendo un ordinamento stabile (in coda ai pari-priorità). Quando incontra una priorità numericamente maggiore, la scansione si ferma: l'indirizzo corrente diventa il next_target_addr e si inizializza un puntatore di shift (next_shift_addr) puntato alla fine della lista, transitando in SHIFT_READ.
- **SHIFT_READ / SHIFT_WRITE:** Questo ciclo effettua lo "shift all'indietro" per fare spazio al nuovo elemento senza perdere dati. Partendo dal fondo della lista, il task in posizione N viene letto e immediatamente riscritto nella posizione N+1. Il puntatore curr_shift_addr viene decrementato ad ogni iterazione. Quando anche l'elemento situato in target_addr è stato traslato liberando la cella, si esce dal loop transitando in INSERT_TASK.
- **INSERT_TASK:** Il sistema concatena i 6 bit dell'ID e i 2 bit della priorità in un unico byte e lo scrive fisicamente nella memoria all'indirizzo curr_target_addr appena liberato. Si transita quindi in UPDATE_TASKS_NUM.
- **UPDATE_TASKS_NUM:** La FSM si posiziona all'indirizzo 0, vi sovrascrive il contatore totale dei task incrementandolo di 1, e chiude l'operazione transitando nello stato DONE.

OPERAZIONE 11 - Svuotamento della lista

- **CLEAR_MEM (OP 11):** Il modulo imposta istantaneamente i segnali per scrivere il valore 0 all'indirizzo 0 della memoria, svuotando logicamente l'intera lista in un solo ciclo di clock, e passa a DONE.

OPERAZIONI DI SUPPORTO - Protocollo di handshake start-done

- **DONE / WAIT_START_DOWN:** Questi stati gestiscono la corretta chiusura dell'operazione. In DONE, viene alzato il segnale o_done a 1 e l'eventuale ID estratto viene esposto in uscita. La FSM attende in questo stato finché il dispositivo esterno non abbassa il segnale i_start. Rilevata la discesa, transita in WAIT_START_DOWN per azzerare o_done e, al ciclo successivo, torna in WAIT_OPERATION in attesa di nuovi comandi.

3. Sintesi e Testbench

Il processo di sintesi logica è stato eseguito tramite l'ambiente di sviluppo Xilinx Vivado 2019.1 impostando come target la scheda FPGA xc7a200tsbg484-1. L'obiettivo di questa

fase è stato verificare che l'approccio comportamentale adottato venisse correttamente tradotto e ottimizzato dal tool di sintesi in componenti fisici elementari.

L'analisi dei report generali a valle della sintesi ha restituito risultati ottimali, evidenziando le seguenti metriche:

- 1) Utilization Report
 - a) LUT (Look Up Tables): sono state impiegate 212 LUT pari allo 0.16% delle risorse disponibili sul chip. Il basso utilizzo di logica combinatoria dimostra che Vivado è riuscito a ottimizzare e condividere le risorse aritmetiche attraverso i diversi stati della macchina
 - b) Flip-Flop: il design utilizza 115 Flip-Flop pari allo 0.4% del totale. Questi registri mappano esattamente il Registro di Stato e i registri operativi del Datapath previsti nel processo sincrono
 - c) I/O ports: sono stati utilizzati 54 pin di I/O per interfacciarsi correttamente con la memoria e i segnali di controllo esterni.
- 2) Timing Report: il sistema è stato vincolato a operare con un clock di sistema avente periodo pari a 20ns. Il timing report conferma il pieno rispetto dei vincoli di temporizzazione:
 - a) Il WNS (Worst Negative Slack) risulta ampiamente positivo ed è pari a 14.998 ns. Questo indica che il percorso combinatorio più lungo (situato probabilmente nella logica di comparazione delle priorità o nel controllo dei duplicati) si risolve con largo anticipo rispetto al fronte di salita successivo del clock.
 - b) Il WHS (Worst Hold Slack) è anch'esso positivo (9,5 ns) , garantendo la stabilità dei dati sui registri.
- 3) Latch: un'attenta analisi dei warning nel tool di sintesi dimostra l'assenza di Latch indesiderati. Questo risultato certifica la correttezza formale della descrizione RTL della rete combinatoria, garantita dalla pratica progettuale di aver fornito assegnazioni di default per tutti i segnali in uscita all'inizio del processo combinatorio

- 1) Testbench 1: Scenario standard (tb fornito)
 - a) **Obiettivo:** verificare il corretto ciclo di vita della lista, simulando tutte le operazioni fondamentali
 - b) **Analisi:** questo test conferma che la FSM gestisce correttamente il protocollo di handshake tramite i_start e o_done e che l'algoritmo di ricerca della posizione e lo shift all'indietro funzionano correttamente nei casi standard. E' stato fondamentale per validare la coerenza del contatore all'indirizzo 0
- 2) Testbench 2: Stress test e casi Limiti
 - a) **Obiettivo:** dimostrare la robustezza del componente a:
 - i) Inserimento di ID duplicati
 - ii) Operazioni (rimozione OP=01 e incremento priorità OP=00) su lista vuota
 - iii) Saturazione della priorità
 - b) **Analisi:**

- i) Il test ha confermato che la fase di DUPLICATE_CHECK impedisce l'inserimento di un task il cui ID è già presente in memoria, evitando la corruzione della lista
- ii) Si è verificato che le operazioni di rimozione e incremento di priorità eseguite con zero task non portino la FSM in stalli o errori di memoria, restituendo correttamente l'ID 00000
- iii) Il test ha validato che i task con priorità 3 non subiscano overflow durante l'operazione 00, rimanendo stabili al valore massimo previsto

4. Conclusioni

A conclusione del progetto, si può affermare che l'architettura sviluppata dimostra un'elevata robustezza strutturale nella gestione dinamica della memoria. L'implementazione di una FSM a singolo modulo ha permesso di ottimizzare le risorse e di gestirle in totale autonomia la compattazione della coda a priorità tramite meccanismi di shift.

Un aspetto rilevante, emerso durante la fase di validazione e analisi dei casi limite, riguarda la gestione della capacità massima della memoria. Il contatore dei task, allocato all'indirizzo 0 su un bus a 8 bit, ha il limite teorico di overflow fissato a 255 elementi. Tuttavia, la rete logica progettata risulta intrinsecamente immune al rischio di saturazione e corruzione della memoria.

L'identificativo dei task (`i_task_id`) è codificato su 6 bit, consentendo un massimo di 64 combinazioni. Escludendo l'ID 000000, convenzionalmente utilizzato per indicare una lista vuota o non valida, il sistema può gestire un massimo di 63 task univoci. Avendo implementato un algoritmo di Pre-Scan per la prevenzione dei duplicati (DUPLICATE_CHECK), il sistema scarterà matematicamente qualsiasi tentativo di inserimento oltre il 63esimo elemento consecutivo, poiché l'ID proposto coinciderà necessariamente con uno dei task già allocati nella coda.

Questa proprietà rende impossibile il verificarsi di un memory overflow o lo sballamento dei puntatori, garantendo al dispositivo una stabilità operativa assoluta anche qualora venisse sollecitato in modo anomalo o incontrollato dall'unità di elaborazione esterna.